

LLMs

Large Language Models

- “Large” - относительное понятие. Размер модели определяется количеством ее параметров. Рассмотрим, например, семейство GPT:
 - GPT-1 - 117 млн. параметров;
 - GPT-2 - 1.5 млрд. параметров;
 - GPT-3 - 175 млрд. параметров;
 - GPT-4 - 1 трлн (?)
- Для сравнения, вариант T5, на основе которого мы обучали модель для упрощения, имеет 244 млн. параметров.

Обучение больших языковых моделей

1. Pre-train

- Очень дорогая стадия, направленная на усвоение как можно большего количества информации из больших объемов данных.

2. Supervised Fine-tuning

- Дообучение базовой модели на конкретные инструкции.

3. Reinforcement Learning [from Human Feedback]

- Модель практикуется в решении задач и поиске оптимального решения.

Pre-training: данные

- Common Crawl (<https://commoncrawl.org/>)
 - A free, open repository of web crawl data that can be used by anyone;
 - Более 250 миллиардов страниц во всём корпусе, собранных из интернета в определенные даты;
 - Страницы могут быть отфильтрованы по определенным критериям: язык, чувствительный контент (есть списки забаненных ресурсов для скачивания) и т.п.

Pre-training: процесс

- Модель обучается предсказывать следующий токен.
- Длина последовательности, которую можно поместить в модель, называется контекстным окном (context window).

Модель, которая обучена только на предсказание следующего токена, называется **базовой (base model)**.

Supervised Fine-Tuning (SFT)

Supervised Fine-Tuning (SFT)

- После того, как модель запомнила достаточно данных и научилась продуцировать язык на их основе, ее можно дообучать на понимание конкретных инструкций.
- Supervised, потому что на этой стадии в модель подаются данные формата “инструкция + вопрос + правильный ответ”.
- На этом этапе модель выучивает определенные токены, которые подсказывают ей, как идет диалог.

Модель, обученная следованию инструкциям, называется **instruct model**.

Tiktokenizer

gpt-4o

System



You are a helpful assistant



User



What is the name of Elizabeth Bennet's love interest in Jane Austen's "Pride and Prejudice"?



Assistant



Fitzwilliam Darcy



Add message

```
<|im_start|>system<|im_sep|>You are a helpful assistant<|im_end|><|im_start|>user<|im_sep|>What is the name of Elizabeth Bennet's love interest in Jane Austen's "Pride and Prejudice"?<|im_end|><|im_start|>assistant<|im_sep|>Fitzwilliam Darcy<|im_end|><|im_start|>assistant<|im_sep|>
```

Token count

47

```
<|im_start|>system<|im_sep|>You·are·a·helpful·assistan  
t<|im_end|><|im_start|>user<|im_sep|>What·is·the·name·  
of·Elizabeth·Bennet's·love·interest·in·Jane·Austen's  
·"Pride·and·Prejudice"?<|im_end|><|im_start|>assistant  
<|im_sep|>Fitzwilliam·Darcy<|im_end|><|im_start|>assis  
tant<|im_sep|>
```

```
200264, 17360, 200266, 3575, 553, 261, 10297, 29186, 2  
00265, 200264, 1428, 200266, 4827, 382, 290, 1308, 32  
8, 31806, 61761, 292, 885, 3047, 3425, 306, 23919, 147  
000, 885, 392, 47, 2037, 326, 4659, 184888, 69029, 200  
265, 200264, 173781, 200266, 37, 9825, 21053, 5866, 18  
8305, 200265, 200264, 173781, 200266
```

☒ Show whitespace

SFT: данные

- Коллекции вопросов и ответов на них, опционально – с системным промптом.
- В идеале ответы должны быть написаны экспертами-людьми по инструкциям. На практике подобные датасеты могут быть частично или полностью сгенерированы другими LLM.

SFT: learning objectives

Чему можно научить модель на этом этапе?

- Отказываться отвечать на вопросы;
- Отвечать на вопросы шаблонно (as a language model, I cannot...);
- Применять функции: например, генерировать токены, которые заставят приложение вызвать функцию веб-поиска;
- Отвечать “не знаю”;
- Генерировать цепочку размышлений;
- ...

Models need tokens to think

Модели не хватает внутренних знаний, чтобы сразу правильно ответить на вопрос. Но она “думает”, симулируя пошаговое выполнение задачи, и тем самым приходит к правильному ответу в процессе генерации.

Этот феномен влияет в том числе на инструкции по подготовке данных для обучения.

USER	how many letters "c" does the word "Pistaccio" have?
ASSISTANT	The word "Pistaccio" has 8 letters: P-I-S-T-A-C-C-I-O.

USER	how many letters "c" does the word "Pistaccio" have? Write Python code.
ASSISTANT	The word "Pistaccio" has 8 letters: P-I-S-T-A-C-C-I-O. Here's a Python code snippet to count the number of 'c' in the word: <pre>```python def count_c(word): """Return the number of 'c' in the given word.""" return word.lower().count('c') word = "Pistaccio" num_c = count_c(word) print(f"The word '{word}' has {num_c} 'c's.") ```</pre> When you run this code, it will output: <pre>``` The word 'pistaccio' has 2 'c's. ```</pre> This code works by converting the word to lowercase (so it treats 'C' and 'c' as the same letter) and then using the 'count' method to find the number of occurrences of 'c'.

Используем знания из параметров (внутри модели)

USER	how many letters "c" does the word "Pistaccio" have?
ASSISTANT	The word "Pistaccio" has 8 letters: P-I-S-T-A-C-C-I-O.

Добавляем знания из контекстного окна

USER	how many letters "c" does the word "Pistaccio" have? Write Python code.
ASSISTANT	The word "Pistaccio" has 8 letters: P-I-S-T-A-C-C-I-O. Here's a Python code snippet to count the number of 'c' in the word: <pre>'''python def count_c(word): """Return the number of 'c' in the given word.""" return word.lower().count('c') word = "Pistaccio" num_c = count_c(word) print(f"The word '{word}' has {num_c} 'C's.") '''</pre> When you run this code, it will output: <pre>''' The word 'pistaccio' has 2 'C's. '''</pre> This code works by converting the word to lowercase (so it treats 'C' and 'c' as the same letter) and then using the 'count' method to find the number of occurrences of 'c'.

Reinforcement Learning

Reinforcement learning - напоминание из самой первой лекции

Агент (модель) взаимодействует с неизвестной ему средой. Он должен выбрать такую стратегию поведения, которая максимизирует награду.

Пример: диалоговые системы

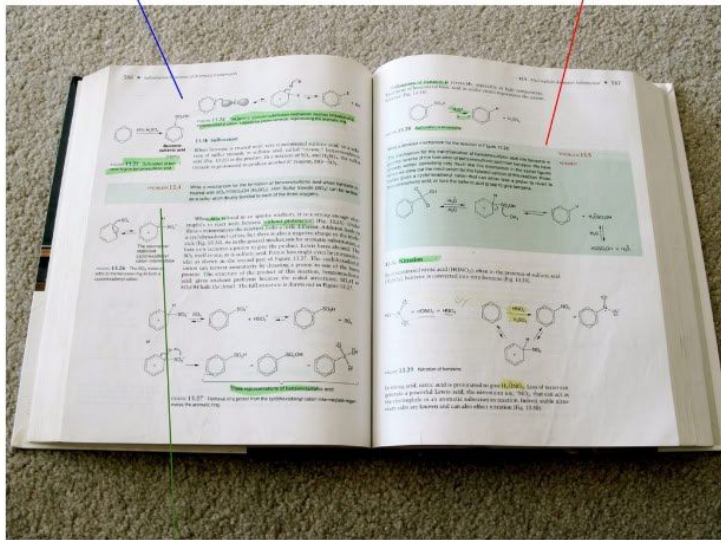
- Наблюдения: запрос от пользователя;
- Действия: генерация ответа;
- Награда: баллы за информативные, неодинаковые и грамматически правильные ответы.

Reinforcement learning

Модель практикуется в решении задач и обучается выбирать лучшее.

exposition $\Leftrightarrow$ pretraining
(background knowledge)

worked problems $\Leftrightarrow$ supervised finetuning
(problem + demonstrated solution, for imitation)



practice problems $\Leftrightarrow$ reinforcement learning
(prompts to practice, trial & error until you reach the correct answer)

Reinforcement learning: вход

- Набор задач: например, вопросов от пользователя;
- Функция награды, которую можно посчитать автоматически (соответствие базе ответов, BLEU, ROUGE, F1 и т.п.)

Reinforcement learning: процесс

Для каждой итерации:

1. Получить входные данные (например, вопрос);
2. Модель генерирует ответ;
3. Оценить ответ с использованием автоматизированных метрик;
4. Рассчитать вознаграждение;
5. Обновить параметры модели на основе вознаграждения.

Reinforcement learning: use cases

- Оптимизация сложных задач: оценка безопасности запроса, ответы на открытые вопросы;
- Когда аннотированных данных мало, но для задачи можно хорошо определить функцию вознаграждения (= verifiable domain);
- Когда нужно выйти за пределы имитационного обучения (SFT).

Reinforcement learning with Human Feedback

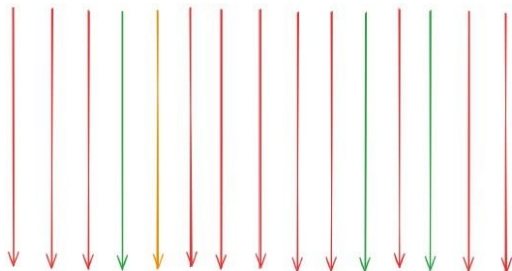
Подходит для областей, где автоматическую оценку сложно или невозможно применить (unverifiable domains) - например, генерация шуток.

- Небольшое количество ответов модели проверяется живыми аннотаторами (например, модель дает по 5 ответов на 100 промптов, эти 500 ответов ранжируются экспертами от 1 до 5);
- В качестве функции вознаграждения выступает другая модель, которая на аннотированных данных учится решать простую задачу оценивания ответов.

RL:

prompt

Emily buys 3 apples and 2 oranges. Each orange costs \$2. The total cost of all the fruit is \$13. What is the cost of each apple?



Answer: 3

We generated 15 solutions.
Only 4 of them got the right answer.
Take the top solution (each right and short).
Train on it.
Repeat many, many times.

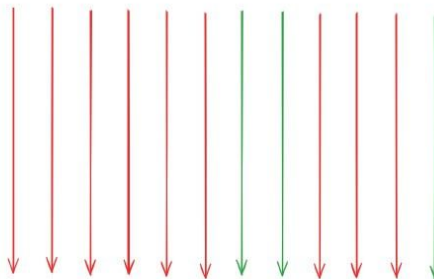
RLHF:

Reinforcement Learning in un-verifiable domains
=> RLHF (Reinforcement Learning from Human Feedback)

Fine-Tuning Language Models from Human Preferences

Daniel M. Ziegler* Nisan Stiennon* Jeffrey Wu Tom B. Brown
Alec Radford Dario Amodei Paul Christiano Geoffrey Irving
OpenAI
(dms,nisan,jeffwu,tom,alec,damodei,paul,irving)@openai.com

prompt:
"write a joke about pelicans"



problem: how we do score these *at scale* (automatically)?

Naive approach:

Run RL as usual, of 1,000 updates of 1,000 prompts of 1,000 rollouts.
(cost: 1,000,000,000 scores from humans)

RLHF approach:

STEP 1:

Take 1,000 prompts, get 5 rollouts, order them from best to worst
(cost: 5,000 scores from humans)

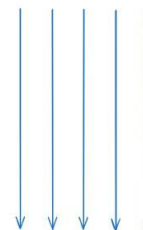
STEP 2:

Train a neural net simulator of human preferences ("reward model")

STEP 3:

Run RL as usual, but using the simulator instead of actual humans

prompt:
"write a joke about pelicans"



reward model scores: 0.1 0.8 0.3 0.4 0.5

human ordering: 2 1 3 5 4

RLHF: проблемы

- Теперь модель вознаграждения можно обмануть. Например, агент (модель, генерирующая ответы на вопросы) может научиться искать ответы, которые объективно не являются лучшими, но при этом максимизируют награду.

Решение: мониторинг данных, мониторинг ответов системы, короткое обучение (около 100 итераций).

Post-training, но побыстрее

Проблемы в обучении LLM

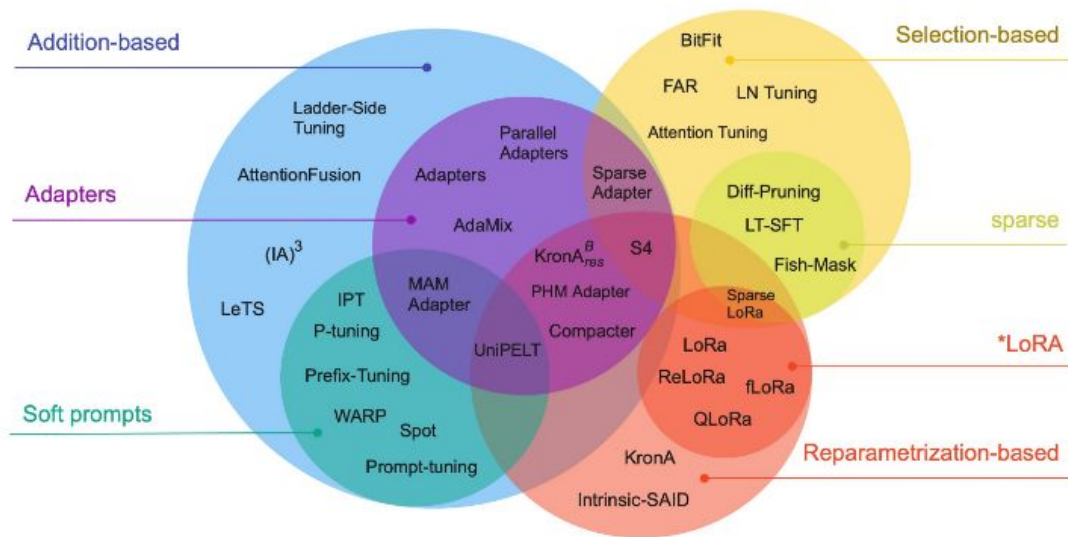
- Из-за большого количества параметров полное дообучение требует больших вычислительных мощностей;
- Forward pass при инференсе также идет долго.

Решения:

- Облегчение моделей;
- Облегчение обучения;
- Ускорение инференса.

Parameter-Efficient Fine-tuning (PEFT)

PEFT - методы эффективного дообучения моделей на downstream-задачи.



PEFT: методы

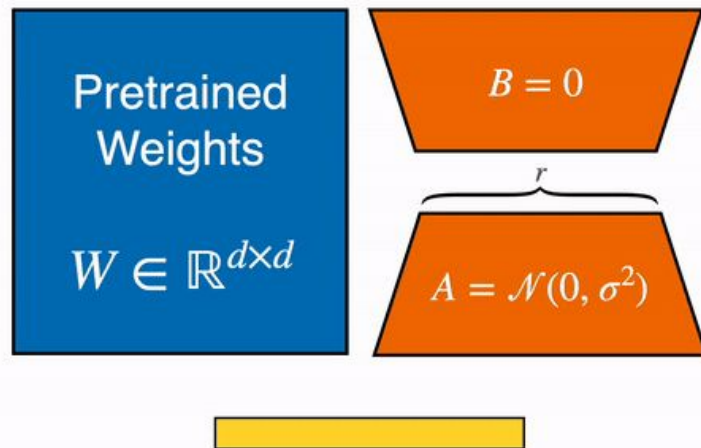
- Аддитивные: в модель добавляются новые параметры и обучаются только они;
 - Например: адаптеры - небольшие полносвязные сети, которые вставляются между определенными слоями трансформера и позволяют адаптироваться к новым задачам без необходимости обновлять все веса;
- Селективные: выбираем, какие параметры обновлять, и обновляем только их. Например, обновляем только байесы, или только последние слои;
- Репараметризация: количество параметров для дообучения сокращается за счет использования низкоранговых матричных представлений.
 - Например: LoRA.

Можно выбрать оптимальный метод для ваших задач: <https://huggingface.co/docs/peft/index>

LoRA: Low-Rank Adaptation of Large Language Models

Идея: декомпозируем веса на две низкоранговые матрицы и дообучаем только их.

Ранг матриц A и B является гиперпараметром.



LoRA: особенности

- Можно хранить адаптеры отдельно, можно смержить с базовой моделью;
- Можно иметь несколько адаптеров под разные задачи, и на этапе инференса просто менять их;
- Можно выбирать модификации алгоритма под конкретные условия - например, выбрать QLoRA для дополнительного уменьшения веса итоговой модели.

Квантизация

Квантизация: идея

Можно поменять формат представления параметров таким образом, чтобы они занимали меньше места в памяти.

Float 32-bit (FP32)

0 10000000 10010010000111111011011

$$(-1)^0 \times 2^1 \times 1.5707964 = 3.1415927410125732$$

higher precision

Float 16-bit (FP16)

0 10000 1001001000

$$(-1)^0 \times 2^1 \times 1.5703125 = 3.140625$$

lower precision

original value
3.1415927

Original Image



"Quantized" Image



Квантизация: методы

- Квантизация в момент обучения (Quantization Aware Training):
 - Во время обучения также учимся квантизировать.
 - Потеря точности меньше, но ресурсов тратится больше.
- Квантизация после обучения:
 - Квантизируем веса и активации готовой модели.
 - Известный метод четырехбитной квантизации для использования моделей на CPU - GGUF.

Можно пользоваться гайдом для выбора подходящего метода для вашей задачи: <https://huggingface.co/docs/transformers/main/en/quantization/overview>

Проблемы

- Деквантизация до оригинальной точности зачастую невозможна;
- Соединение LoRA-адаптеров с квантизированными моделями может привести к проблемам с качеством инференса (<https://github.com/huggingface/peft/issues/2321>).

Как еще можно ускорить обучение?

- Если у вас несколько видеокарт: воспользоваться решениями для распределения вычислений
 - 🧐 Accelerate - библиотека для распределения вычислений на GPU/TPU, работает с любой моделью на pytorch
- Воспользоваться 🧐 flash attention для оптимизации вычислений на слоях внимания.
 - Подходит как для обучения, так и для инференса.

Ускорение инференса

Идея

В production-level решениях очень важна скорость генерации.
Следовательно, необходимы инструменты для ускорения инференса.

Популярные библиотеки: vLLM, TGI

Как это работает

- Манипуляции и улучшения механизма внимания: PagedAttention в vLLM, Flash Attention-2 в TGI;
- Работа с памятью: динамическое выделение блоков памяти, эффективное заполнение видеокарты
- Некоторые инструменты поддерживают, например, быструю замену адаптеров над базовой моделью (lorax).

О чем стоит помнить

- Качество инференса может измениться после применения акселератора;
- Возможность применения какого-либо инструмента может зависеть от того, поддерживает он использование вашей базовой модели или нет, и насколько хорошо.

Практика

1. Gigachat API:

https://colab.research.google.com/drive/1GvdKNv_LALdwB5wAAOjXMpifl4KQ6ZE2?usp=sharing

2. Unsloth:

https://colab.research.google.com/drive/1xawAaigreeft7B1mbm68_iVmRC2ZYQW3?usp=sharing

3. LLaMA-Factory:

https://colab.research.google.com/drive/18aReoN_GaSjivxSsYt2ZUuNTYdOjEXx5?usp=sharing

Проверочные работы

1. Квиз №3:

<https://docs.google.com/forms/d/e/1FAIpQLSeektRI30hMUNI0xB1Sz3eJ3Lmk3qlludsF87YyNTLonfVSZw/viewform?usp=dialog>

2. Лабораторная №2 - дедлайн 30 декабря 23.59 **МСК**.

ИСТОЧНИКИ

1. Тutorials Andrej Karpathy:

a. <https://youtu.be/7xTGNNLPyMI?si=yQRtwvRj-mkM6zjz>

b. <https://youtu.be/EWvNQjAaOHw?si=3Y0-mw9GfUFBNTEP>

2. Tiktokenizer: <https://tiktokenizer.vercel.app/>

3. PEFT: <https://huggingface.co/blog/samuellimabraz/peft-methods>

4. Квантизация:

<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization>

Спасибо за внимание!